

MAKERERE

P.O.Box 7062 Kampala Uganda
e-mail: mech@starcom.co.ug



UNIVERSITY

Phone: 041-541173 Fax: 041-530686
Cables: "MAKUNIKA"

COLLEGE OF COMPUTING AND INFORMATION SCIENCES

SCHOOL OF COMPUTING AND INFORMATICS TECHNOLOGY

BACHELOR OF SCIENCE IN SOFTWARE ENGINEERING

DESIGN DOCUMENT FOR SMART DISCUSSION FORUM

GITHUB LINK : <https://github.com/OumoSteven/Smart-discussion-forum>

Group number -28

SUPERVISOR:DR EPHRANCE NAMUGENYI

Name	Registration No.	Student No.
OUMO STEVEN	25/U/30750	2500730750
KAYENGERE BRENDA	25/U/0383	2500700383
NAMUWENDO JUDITH MUKISA	25/U/0389	2500700389
NALUBWAMA JOAN JOJO	25/U/03283/PS	2500703283
BUKENYA ALPHA DANIEL	25/U/04192/PS	2500704192

Table of content

Table of content.....	2
List of Table.....	6
List of figures.....	7
1. Introduction.....	8
1.1 Purpose.....	8
1.2 System overview	8
1.3 System requirements	8
1.3.1 Functional requirements	8
1.3.2 Nonfunctional requirements	9
1.4 Intended audience.....	10
1.5 References	10
2. Assumptions and Dependencies.....	10
2.1 Assumptions.....	10
2.2 Dependencies.....	11
2.3 Constraints.....	12
2.4 Machine learning design assumptions.....	12
3. System architecture.....	13
3.1 Architectural pattern.....	13
3.2 Architecture diagram.....	13
3.3Architecture description	13
3.4 Design rationale.....	16
4. Database Design.....	17
4.1 Conceptual data model.....	17
4.2 Entity descriptions.....	17

4.3 Data dictionary	18
4.4 Storage and persistence strategy	23
4.5 Data integrity and retention	24
5. Component Design	25
5.1 Overview	25
5.2 Component Diagram	25
5.3 API Gateway and Authentication Component	26
Purpose	26
Responsibilities	26
Operations	26
Interactions	26
5.4 Membership Management Component	26
Purpose	26
Responsibilities	26
Operations	27
Interactions	27
5.5 Discussion Management Component	27
Purpose	27
Responsibilities	27
Operations	27
Interactions	28
5.6 Moderation Component	28
Purpose	28
Responsibilities	28
Operations	28
Interactions	28
5.7 Notification Component	28

Purpose.....	28
Responsibilities.....	28
Operations.....	29
Interactions.....	29
5.8 Quiz Engine Component.....	29
Purpose.....	29
Responsibilities.....	29
Operations.....	29
Interactions.....	30
5.9 Grading Component.....	30
Purpose.....	30
Responsibilities.....	30
Operations.....	30
Interactions.....	30
5.10 Analytics Component.....	30
Purpose.....	30
Responsibilities.....	30
Operations.....	31
Interactions.....	31
5.11 Synchronization Component.....	31
Purpose.....	31
Responsibilities.....	31
Operations.....	31
Interactions.....	31
5.12 Sharing and Export Component.....	32
Purpose.....	32
Responsibilities.....	32

Operations.....	32
Interactions.....	32
5.13 Machine Learning Recommendation Component.....	32
Purpose.....	32
Responsibilities.....	32
Operations.....	32
Interactions.....	33
5.14 Selected component behaviour.....	33
6. Interface design.....	34
6.1 Overview	34
6.2 REST API (representative endpoints).....	34
6.3 Real-time channels (WebSocket).....	35
6.4 ML service API.....	36
6.5 Internal interfaces.....	36
6.6 Hardware and software interfaces.....	37
6.7 Communication protocols.....	37
7. Security Design.....	37
7.1 Overview	37
7.2 Authentication.....	37
7.3 Authorization and Access Control.....	38
7.4 Secure Communication	38
7.5 Data Protection.....	38
7.6 Database Security.....	39
7.7 Logging and Monitoring.....	39
7.8 Backup and Recovery.....	40
8. User Interface Design.....	41
8.1 Login interface.....	41

8.2 Student Dashboard.....	42
8.3 Lecturer Dashboard.....	43
8.4 Admin Dashboard.....	44
9. Deployment architecture.....	45
9.1 Overview.....	45
9.2 Deployment Environment.....	45
9.3 Deployment diagram.....	45
9.4 Deployment Components.....	45
9.5 Communication Between Components.....	46
9.6 Deployment Rationale.....	47
9.7 Risk register.....	47
10. Glossary of Terms.....	48

List of Table

Table 2. 1 dependencies Table.....	12
Table 4. 1 User Table.....	19
Table 4. 2 Member table.....	19
Table 4. 3 Topic table.....	20
Table 4. 4 Post Table.....	21
Table 4. 5 Quiz Table.....	21
Table 4. 6 Question table.....	22
Table 4. 7 Quiz Attempt table.....	22
Table 4. 8 marks table.....	23
Table 4. 9 Additional entities.....	23
Table 6. 1 REST API Table.....	35
Table 6. 2 Table for Real-time channels.....	35
Table 6. 3 Table for ML service API.....	36

Table 6. 4 Table for Internal interfaces.....	36
---	----

Table 9. 1: Communication table.....	47
--------------------------------------	----

Table 9. 2: Risk register.....	47
--------------------------------	----

Table 10. 1: Table of Glossary of Terms.....	50
--	----

List of figures

Figure 3. 1 System Architecture of the Smart Discussion Forum.....	13
--	----

Figure 4. 1: conceptual diagram of the Smart Discussion Forum.....	17
--	----

Figure 5. 1 component Design.....	25
-----------------------------------	----

Figure 8. 1: Diagram of the login screen.....	41
---	----

Figure 8. 2: Diagram of the student dashboard screen.....	42
---	----

Figure 8. 3: Diagram of the lecture dashboard screen.....	43
---	----

Figure 8. 4: Diagram of the Admin dashboard screen.....	44
---	----

1. Introduction

1.1 Purpose

This Software Design Document (SDD) describes the architecture and design of the Smart Discussion Forum. It serves as a blueprint for the development team and stakeholders by providing detailed specifications of the system's functionality, architecture, components, requirements and interfaces.

1.2 System overview

The Smart Discussion Forum is a managed discussion and learning platform for academic groups such as classes and research teams. It is delivered through two clients that share a single backend that is a web client built with Laravel that provides online access to all system features through a browser and a desktop client built with a Java Graphical User Interface (GUI) that supports both online and offline operations. The machine-learning component enhances the platform by providing topic classification, filtering irrelevant content, identifying unanswered questions and recommending relevant discussions to users.

When a user requests to join a group, the Membership service creates a membership in a pending state and the onboarding flow requires the member to accept the group rules before participating. The service records activity on each meaningful action. A scheduled check compares the last-active time against the inactivity threshold the first two breaches raise warnings through the Notification service and a third places the member on a timed blacklist by setting the membership status and a blacklist-until time. The blacklist clears automatically once that time passes.

A lecturer configures a quiz with a start time and duration through the Quiz Engine. At the start time the quiz opens and members begin attempts in a locked-down interface. The server, not the client, is the timing authority. When the duration elapses the attempt is auto-submitted and members who join late receive no additional time because the deadline is anchored to the quiz. After closing, the Quiz Engine grades attempts and produces a performance report.

1.3 System requirements

1.3.1 Functional requirements

- The Smart discussion forum system shall provide the following functionalities;

- User registration and onboarding. The system shall allow new users to register and require them to read and accept platform rules before participation
- Discussion management. The system shall allow users to create topics, post messages, reply to discussions and view conversations.
- Content filtering. The system shall automatically identify and filter irrelevant discussion posts.
- Unanswered question detection. The system shall identify unanswered questions and highlighted available answers when posted.
- Communication restriction. The system shall allow administrators to restrict selected members from participating in some communications.
- Inactivity monitoring. The system shall monitor member activity, issue warning notifications and automatically blacklist inactive users.
- PDF export. The system shall allow users to export topic discussions to PDF format
- Statistical reporting. The system shall generate participation statistics and activity reports for administrators.
- Offline synchronization. The desktop application shall support offline access and synchronize data when internet is restored.
- Participation assessment. The system shall calculate and award participation marks according to predefined criteria.
- Quiz management. The system shall allow lecturers to create, schedule, administer and evaluate quizzes.
- Topic classification. The system shall use machine learning techniques to classify discussion topics automatically
- Topic recommendation. The system shall recommend relevant discussion topics based on user's previous interests.
- Social media sharing. The system shall allow sharing of discussion posts on supported social media platforms

1.3.2 Nonfunctional requirements

- Performance. The system shall provide real-time message delivery and respond to user requests within acceptable response times.
- Reliability. The system shall ensure data consistency and successful synchronization.
- Availability. The system shall be accessible to users at all times.
- Security. The system shall provide user authentication and role-based access control.

- Usability. The system shall provide a friendly user interface that requires minimal training
- Scalability. The system shall support increasing numbers of users, discussion groups without performance degradation
- Maintainability. The system shall adopt a modular architecture to simplify, maintenance and future changes.
- Compatibility. The system shall support modern web browsers and Java-supported desktop environments.
- Data integrity. The system shall ensure that discussion data, quiz records and participation marks remain accurate and consistent.
- The system shall provide mechanisms for data backup and recovery in case of system failure.

1.4 Intended audience

The primary audience is the project development team, who will use this document as the implementation reference. The secondary audience is the project supervisors and assessors, who will use it to evaluate the soundness of the design against the stated requirements.

1.5 References

- Smart Discussion Forum — recess assignment brief (functional requirements).
- IEEE Std 1016, Recommended Practice for Software Design Descriptions.
- Laravel documentation, <https://laravel.com/docs>
- Java documentation, [Java Documentation - Get Started](#)
- MySQL documentation, <https://dev.mysql.com/doc>
- SQLite documentation, <https://sqlite.org/docs.html>
- Redis documentation, <https://redis.io/docs>
- Scikit learn documentation, <https://scikit-learn.org>
- Fast API documentation, [FastAPI - FastAPI](#)
- PDF export libraries, <https://pdfaid.com> , <https://joyfill.io>

2. Assumptions and Dependencies

This section records the conditions that are assumed to hold for the design to be valid, the external components the system depends on and the project constraints that shaped the design decisions.

2.1 Assumptions

- Users are familiar with everyday messaging applications, so the interaction model can follow common chat conventions.
- The web client is accessed from a reasonably modern browser that supports WebSocket connections.
- The desktop client runs on a machine with a supported Java runtime installed.
- Device clocks are reasonably accurate, which the quiz timing and auto-submission logic rely on; the server clock is treated as authoritative.
- Sufficient historical post data is or will become, available to train the topic-classification and recommendation models; until then the system operates with reduced ML accuracy.
- A reachable e-mail (SMTP) service is available for notifications and announcements.
- The targeted social-media platforms expose developer APIs that permit posting on behalf of an authorised account.

2.2 Dependencies

The system depends on the following external frameworks, services and runtimes:

Dependency	Used by	Purpose
Laravel (PHP) + web server	Backend / web client	Application framework, REST API, MVC web pages
Vue	Web client	Interactive front-end components
Java runtime (JDK 17+)	Desktop client	Executes the desktop GUI application
SQLite	Desktop client	Local offline data store
MySQL / PostgreSQL	Data tier	Authoritative relational database

Redis	Data tier	Cache, job queue and publish/subscribe channel
Laravel Reverb (WebSocket)	Realtime	Real-time broadcasting to clients
Python / FastAPI + scikit-learn	ML service	Topic classification and recommendation
SMTP provider	Notification	Outbound e-mail for alerts and announcements
Social-media developer APIs	Sharing & export	Forwarding posts to social platforms

Table 2. 1 dependencies Table

2.3 Constraints

- The development team is limited to five members, so the design favours a small number of well-separated components that individuals can own.
- The SDD is due on 25 June 2026 and final presentations run from 27 to 31 July 2026, which bounds the implementation effort.
- Both a Laravel web application and a Java GUI desktop application must be delivered against the same backend.
- Free hosting is encouraged, so the architecture must run on modest, low-cost infrastructure.

2.4 Machine learning design assumptions

- Sufficient historical data will be available
- Topic classifications improve as more discussions are collected
- Recommendation quality depends on user activity history
- Models will be retrained periodically
- Machine learning service failures should not stop normal forum operations

3. System architecture

3.1 Architectural pattern

The Smart Discussion Forum adopts a layered architecture pattern consisting of four layers that is the presentation layer, application layer, business layer and the data layer. It also uses a Model View Controller design pattern. The architecture separates user interface concerns, business logic, application logic and data management responsibilities. The architectural pattern was selected because it promotes modularity, maintainability, scalability, security and ease of testing. It allows independent modification of individual layers without affecting other components of the system. The architecture also integrates external services to extend the platform's capabilities.

3.2 Architecture diagram

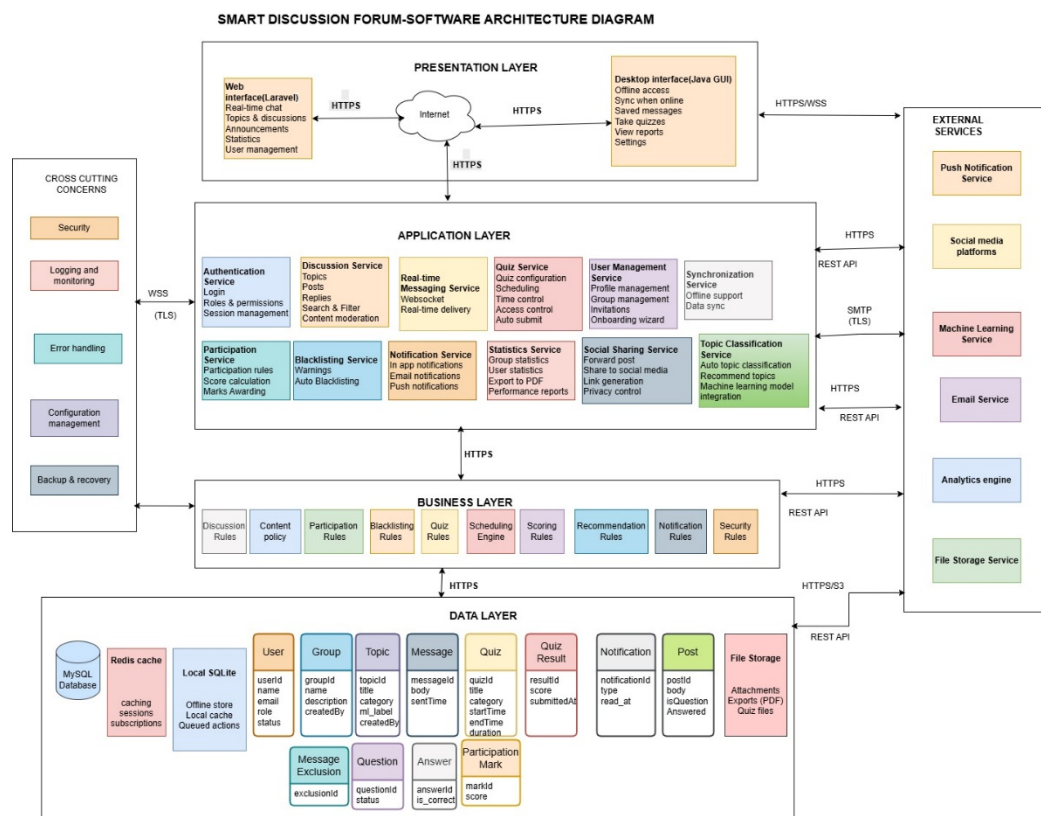


figure 3. 1 System Architecture of the Smart Discussion Forum

HTTPS – Hypertext Transfer Protocol Secure

TLS – Transport Layer Security

REST API- Representational State Transfer Application Programming Interface

WSS- Web socket Secure

SMTP – Simple Mail Transfer Protocol

S3- Simple Storage Service

TCP – Transmission Control Protocol

API- Application Programming Interface

3.3Architecture description

Presentation layer provides user interfaces through which users interact with the system that is;

- Web interface (Laravel) which allows users to participate in discussions, access real-time chat, view announcements, view statistics and manage user profiles.
- Desktop interface (Java GUI) which allows users to access saved discussions offline, synchronize messages when internet is restored, participate in quizzes, view reports and managing application settings.
- Both interfaces communicate with the backend services through the internet.

Application layer contains the core system services responsible for implementing the functional requirements that is;

- Authentication Service which is responsible for user login, session management, role and permission management.
- Discussion Service which is responsible for topic creation, post management, replies and content filtering and moderation.
- Real-Time Messaging Service which is responsible for web socket, communication and Real-time message delivery
- Quiz Service which is responsible for quiz configuration, quiz scheduling, access control and automatic submission
- User Management Service which is responsible for user profiles, group management, invitations, onboarding process
- Synchronization Service which is responsible for offline support and data synchronization between desktop and server
- Participation Service which is responsible for participation tracking, score calculation and awarding participation marks.

- Blacklisting Service which is responsible for inactivity monitoring, warning generation and automatic blacklisting
- Notification Service which is responsible for in-app notifications, email notifications and push notifications
- Statistics Service which is responsible for activity reports, user statistics and PDF report generation
- Social Sharing Service which is responsible for forwarding discussions to social media, link generation and privacy control
- Topic Classification Service which is responsible for automatic topic classification, topic recommendations and machine learning integration

Business Layer contains the business rules governing system operations as per the request. This layer ensures that all system activities follow organizational policies and requirements. These include:

- Discussion Rules
- Content Policies
- Participation Rules
- Blacklisting Rules
- Quiz Rules
- Scheduling Rules
- Scoring Rules
- Recommendation Rules
- Notification Rules
- Security Rules

The Data Layer manages persistent storage and uses a MySQL database to store information about users, messages, notifications and other data.

Cross-Cutting Concerns. Several services affect all layers of the system that is security, logging and monitoring, error handling, configuration management and backup and recovery.

External Services. The system integrates with several external services such as;

- Push Notification Service that delivers real-time notifications to users.
- Social Media Platforms that support sharing discussions externally.
- ML Service that provides topic classification and recommendation capabilities.
- Email Service that sends warnings, notifications and announcements.
- Analytics Engine that generates advanced usage statistics.
- File Storage Service that stores uploaded files and exported documents.

4. Database Design

4.1 Conceptual data model

The data model is centred on users who belong to groups through memberships. Within a group, members create topics that contain threaded posts and exchange group messages. Lecturers configure quizzes made up of questions; members make quiz attempts that record answers. Participation marks and notifications are associated with users. The entity–relationship diagram below shows the entities and the relationships between them.

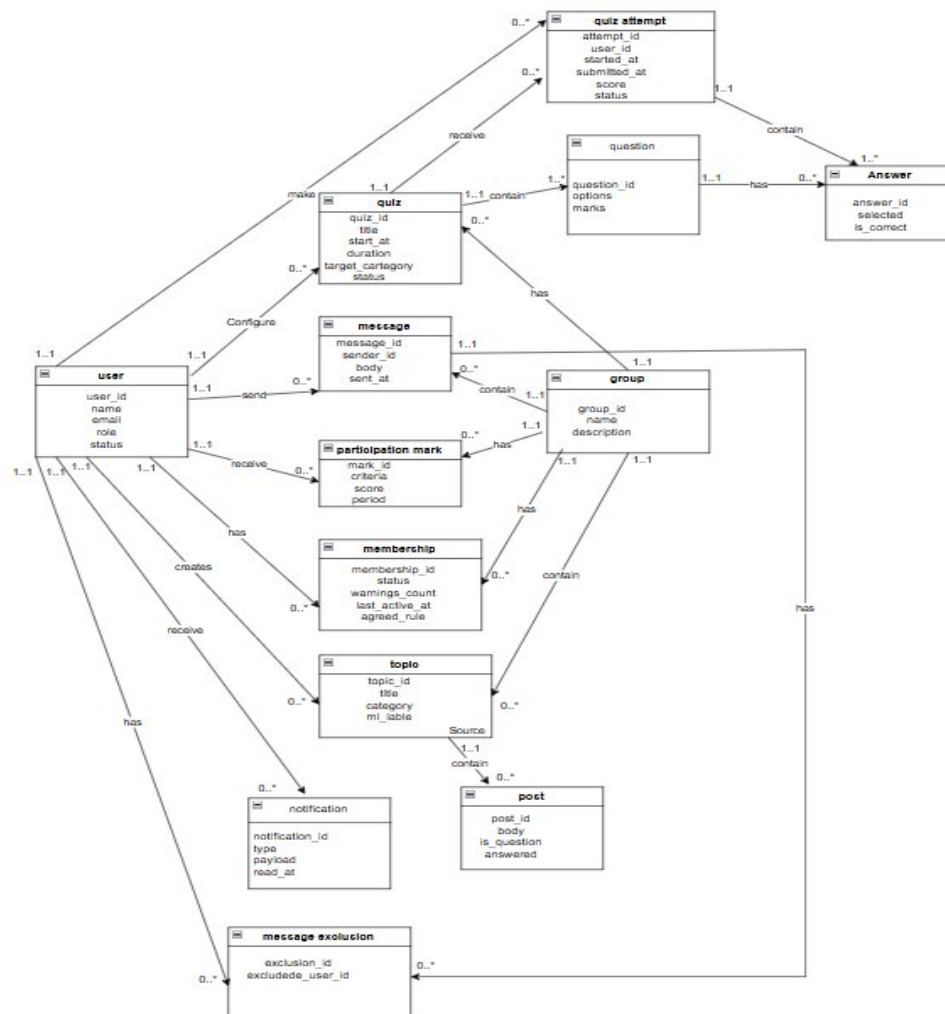


figure 4. 1: conceptual diagram of the Smart Discussion Forum

4.2 Entity descriptions

- **User** — A person who uses the system, in the role of administrator, lecturer or member.
- **Group** — A discussion space such as a class or research team.

- **Membership** — The association of a user with a group; it also carries the activity, warning, rules-agreement and blacklist state used by the membership rules.
- **Topic** — A subject of discussion within a group, with an optional machine-assigned category.
- **Post** — A message inside a topic thread; a post may reply to a parent post, may be a question and may be marked as the accepted answer.
- **Message** — A direct group-chat message, carrying a synchronization status for offline support.
- **MessageExclusion** — Records that a specific user is excluded from a specific message or announcement.
- **Quiz / Question** — A scheduled assessment and its individual questions.
- **QuizAttempt / Answer** — A member's attempt at a quiz and the answers given to each question.
- **ParticipationMark** — A mark awarded to a user for participation, against a named criterion.
- **Notification** — An alert or announcement delivered to a user.

4.3 Data dictionary

The tables below define the principal entities. Types are indicative; PK denotes a primary key and FK a foreign key.

User

FEILD	TYPE	DESCRIPTION
user_id	BIGINT, PK	Unique identifier, Not null
name	VARCHAR(120)	Full display name, Not null
email	VARCHAR(160), unique	Login identity and contact address , Not null
password hash	VARCHAR(255)	Hashed credential , Not null
role	ENUM(admin, lecturer,	System-wide role , Not Null

	member)	
status	ENUM(active, suspended)	Account status , Not Null

Table 4. 1 User Table

Membership

FEILD	TYPE	DESCRIPTION
membership_id	BIGINT, PK	Unique identifier, Not Null
user_id	BIGINT, FK → User	The member , Not Null
group_id	BIGINT, FK → Group	The group joined , Not Null
status	ENUM(active, warned, blacklisted)	Enforcement state , Not Null
warnings_count	INT	Number of inactivity warnings issued (0–2), Not Null
last_active_at	TIMESTAMP	Time of last activity, used to detect inactivity , Not Null
agreed_rules	BOOLEAN	Whether the member accepted the group rules ,Not Null
blacklist_until	TIMESTAMP, nullable	End of a timed blacklist period , Not Null
joined_at	TIMESTAMP	Time the member joined , Not Null

Table 4. 2 Member table

4 Topic

FEILD	TYPE	DESCRIPTION
-------	------	-------------

topic_id	BIGINT, PK	Unique identifier , Not Null
group_id	BIGINT, FK → Group	Owning group , Not Null
created_by	BIGINT, FK → User	Author of the topic , Not Null
title	VARCHAR(200)	Topic title , Not Null
category	VARCHAR(80)	Manually assigned category, if any , Not Null
ml_label	VARCHAR(80)	Category assigned by the classifier , Not Null
is_resolved	BOOLEAN	Whether an answer has been accepted , Not Null
created_at	TIMESTAMP	Creation time , Not Null

Table 4.3 Topic table

Post

FEILD	TYPE	DESCRIPTION
post_id	BIGINT, PK	Unique identifier , Not Null
topic_id	BIGINT, FK → Topic	Owning topic , Not Null
author_id	BIGINT, FK → User	Author of the post, Not Null
parent_post_id	BIGINT, FK → Post, nullable	Parent for threading (self-reference) , Not Null
body	TEXT	Post content , Not Null
is_question	BOOLEAN	Marks the post as a question , Not Null

is_answer	BOOLEAN	Marks the post as the accepted answer , Not Null
relevance_score	FLOAT	Moderation score used to filter irrelevant/flooded posts , Not Null
created_at	TIMESTAMP	Creation time , Not Null

Table 4. 4 Post Table

Quiz

FEILD	TYPE	DESCRIPTION
quiz_id	BIGINT, PK	Unique identifier , Not Null
group_id	BIGINT, FK → Group	Group the quiz belongs to , Not Null
lecturer_id	BIGINT, FK → User	Lecturer who configured the quiz , Not Null
title	VARCHAR(200)	Quiz title , Not Null
start_at	TIMESTAMP	Scheduled start time , Not Null
duration_minutes	INT	Allowed duration , Not Null
target_category	VARCHAR(80)	Topic category the quiz covers , Not Null
status	ENUM(scheduled, open, closed)	Lifecycle state , Not Null

Table 4. 5 Quiz Table

Question

FIELD	TYPE	DESCRIPTION
question_id	BIGINT, PK	Unique identifier , Not Null

quiz_id	BIGINT, FK → Quiz	Owning quiz , Not Null
text	TEXT	Question text, Not Null
options	JSONQue	Answer options , Not Null
correct_option	VARCHAR(10)	Key of the correct option , Not Null
marks	INT	Marks awarded for a correct answer , Not Null

Table 4. 6 Question table

QuizAttempt

FIELD	TYPE	DESCRIPTION
attempt_id	BIGINT, PK	Unique identifier , Not Null
quiz_id	BIGINT, FK → Quiz	Quiz attempted , Not Null
user_id	BIGINT, FK → User	Member attempting , Not Null
started_at	TIMESTAMP	When the attempt began , Not Null
submitted_at	TIMESTAMP, nullable	When it was submitted , Not Null
score	FLOAT	Computed score, Not Null
status	ENUM(in_progress, submitted, auto_submitted)	How the attempt ended, Not Null

Table 4. 7 Quiz Attempt table

Participation Mark

FIELD	TYPE	DESCRIPTION
-------	------	-------------

mark_id	BIGINT, PK	Unique identifier , Not Null
user_id	BIGINT, FK → User	Member marked , Not Null
group_id	BIGINT, FK → Group	Group context , Not Null
criteria	VARCHAR(120)	Criterion the mark is awarded against, Not Null
score	FLOAT	Mark value , Not Null
period	VARCHAR(40)	Period the mark covers , Not Null

Table 4. 8 marks table

Additional entities

FIELD	TYPE	DESCRIPTION
Group	group_id (PK), name, created_by (FK)	A discussion space, Not Null
Message	message_id (PK), group_id, sender_id, body, sync_status	Group-chat message with offline sync state , Not Null
Message Exclusion	exclusion_id (PK), message_id (FK), excluded_user_id (FK)	Excludes a user from a message , Not Null
Answer	answer_id (PK), attempt_id (FK), question_id (FK), selected, is_correct	A member's answer to a question , Not Null
Notification	notification_id (PK), user_id (FK), type, payload, read_at	Alert or announcement to a user , Not Null

Table 4. 9 Additional entities

5. Component Design

5.1 Overview

The Smart Discussion Forum is divided into several independent components. Each component performs a specific responsibility and communicates with other components through well-defined interfaces. This modular approach improves maintainability, scalability and system reliability. The major components are Membership Management, Discussion Management, Moderation, Notifications, Quiz Engine, Grading, Analytics, Synchronization, Sharing and Export and the Machine Learning Recommendation Engine.

5.2 Component Diagram

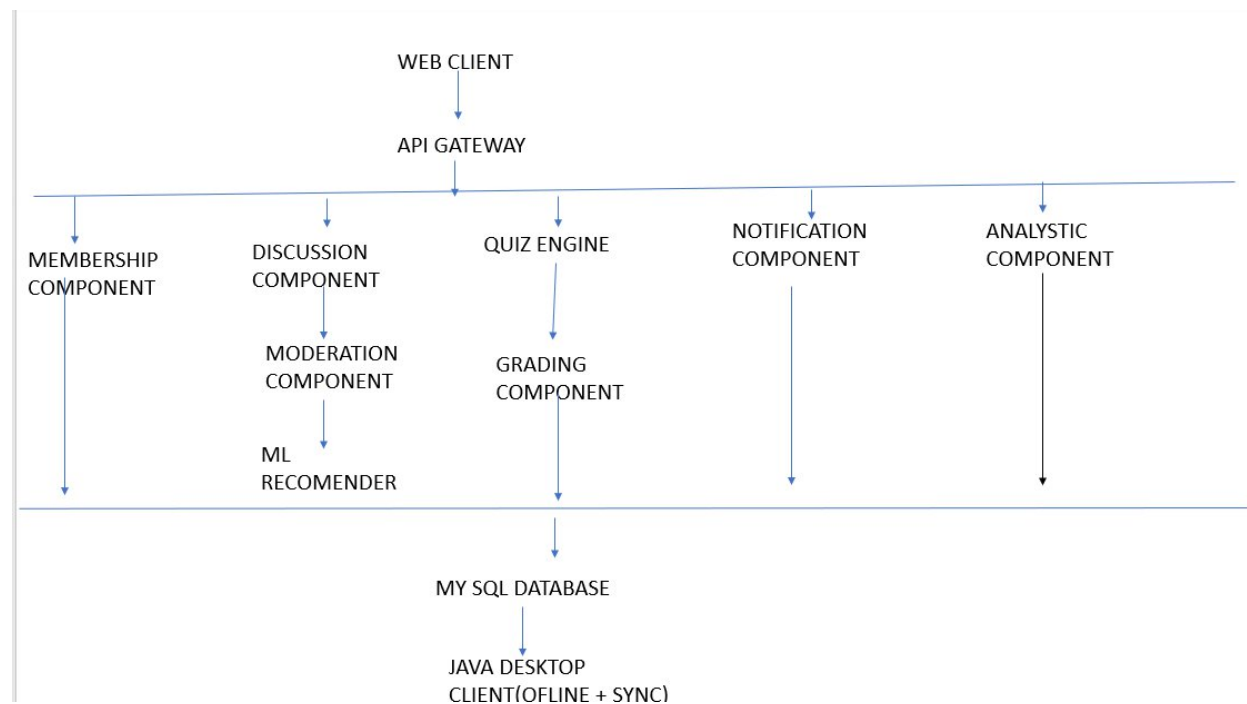


figure 5. 1 component Design

The component diagram illustrates the major modules of the Smart Discussion Forum and the relationships between them. Users access the system through either the Laravel web application or the Java desktop application. Requests are routed through the API Gateway and Authentication component before reaching the internal business components. The business components interact with the database to store and retrieve information while collaborating with one another to satisfy system requirements.

5.3 API Gateway and Authentication Component

This component acts as the entry point into the system. It handles user authentication, authorization and access control. Its responsibilities are user login and logout, user authentication, token generation, permission verification and session management

It will perform the login(), issueToken() and authorize operations. This component communicates with all other system components because every request must first be authenticated before access is granted.

5.4 Membership Management Component

The Membership Management Component manages user registration, onboarding, participation monitoring, warning generation, and blacklisting of inactive users.

This component handles the entire membership lifecycle by processing join requests, recording rule acceptance, tracking user activity, issuing inactivity warnings, maintaining blacklist records, and communicating with the Notification and Grading components to support user engagement and participation assessment.

5.5 Discussion Management Component

The Discussion Management Component manages all discussions within the platform. This component facilitates topic creation, question posting, reply management, answer marking, and thread organization while collaborating with the Moderation Component to ensure content quality and with the Notification Component to generate discussion-related alerts.

5.6 Moderation Component

The Moderation Component maintains the quality of discussions by filtering spam and irrelevant content.

This component evaluates discussion content by detecting spam, controlling flooding behavior, calculating relevance scores, filtering inappropriate posts, and interacting with the Machine Learning Component to improve intelligent content classification and moderation decisions.

5.7 Notification Component

This component delivers alerts, announcements, and warnings to users.

The Notification Component generates and distributes discussion alerts, quiz notifications, warning messages, and administrative announcements while coordinating with other system components to ensure timely communication with users.

5.8 Quiz Engine Component

The Quiz Engine Component manages all assessment-related activities.

This component supports quiz creation, scheduling, delivery, automatic submission, grading, and report generation while interacting with the Notification, Grading, Analytics, and Database components to manage the complete quiz lifecycle.

5.9 Grading Component

The Grading Component automatically calculates participation marks and assessment scores.

This component defines grading criteria, computes participation marks, evaluates quiz performance, generates final scores, and utilizes activity data obtained from discussions and quiz attempts to assess learner performance.

5.10 Analytics Component

The Analytics Component generates reports and statistical summaries.

This component collects data from multiple system modules to analyze user activity, participation trends, discussion performance, and quiz outcomes, producing reports and visual summaries for administrators and lecturers.

5.11 Synchronization Component

The Synchronization Component supports offline-first functionality for the desktop application.

This component manages offline data storage, queues user actions, synchronizes changes with the central server, resolves conflicts during data exchange, and ensures consistency between the Java desktop client and the central database.

5.12 Sharing and Export Component

This component enables sharing and exporting of discussion content.

The Sharing and Export Component generates PDF exports, supports social media sharing, archives discussions, and retrieves discussion information from the Discussion Management Component for distribution and storage.

5.13 Machine Learning Recommendation Component

This component provides intelligent recommendations and topic classification.

This component analyzes discussions, user activities, and participation records to classify topics, generate personalized recommendations, identify user preferences, and continuously improve recommendation quality through model training.

6. Interface design

6.1 Overview

This section describes the interfaces the system exposes and consumes.

6.2 REST API (representative endpoints)

All endpoints are served under /API, exchange JSON and require a bearer token except where marked public. Access is further restricted by role and group membership.

Method & path	Purpose	Access
POST /auth/login	Authenticate and obtain a token	public
POST /groups/{id}/join	Request to join a group (starts onboarding)	member
POST /groups/{id}/agree-rules	Record acceptance of group rules	member
GET /groups/{id}/topics	List topics in a group	member
POST /topics	Create a topic	member
GET /topics/{id}/posts	Get the threaded posts of a topic	member
POST /topics/{id}/posts	Reply within a topic	member
POST /posts/{id}/mark-answer	Mark a reply as the accepted answer	asker / admin
GET /topics/{id}/export	Export the thread to PDF	member
POST /posts/{id}/share	Forward a post to social media	member
POST /messages	Send a group message	member
POST /messages/{id}/exclude	Exclude members from a message	admin
GET /groups/{id}/stats	Retrieve per-group statistics	admin
GET /groups/{id}/flags	List inactivity / blacklist status	admin
POST /quizzes	Configure a quiz	lecturer
POST /quizzes/{id}/attempts	Start a quiz attempt	member

Method & path	Purpose	Access
POST /attempts/{id}/submit	Submit or auto-submit an attempt	member
GET /quizzes/{id}/report	Get the quiz performance report	lecturer
GET /sync/changes?since=	Pull server changes for offline sync	member
POST /sync/push	Push actions queued while offline	member

Table 6. 1 REST API Table

6.3 Real-time channels (WebSocket)

Channel	Events	Purpose
private-group.{id}	post.created, message.sent	Live discussion and chat updates
presence-group.{id}	member.online, member.offline	Online presence in a group
private-user.{id}	notification.created	Personal alerts and announcements
private-quiz.{id}	quiz.opened, quiz.tick, quiz.closed	Quiz start, timing and close events

Table 6. 2 Table for Real-time channels

6.4 ML service API

Method & path	Request	Response
POST /classify	{ text }	{ category, confidence }
POST /moderate	{ text }	{ is_relevant, is_flood, score }
POST /recommend	{ user_id, context }	[topic_id, ...]

Method & path	Request	Response
POST /train	(batch trigger)	{ status, model_version }

Table 6. 3 Table for ML service API

6.5 Internal interfaces

From	To	Mechanism	Data exchanged
API Gateway	Application services	In-process service calls	Validated requests, authenticated user
Application services	Relational DB	Eloquent ORM	Entity reads and writes
Application services	Redis	Cache / queue / pub-sub client	Cached values, jobs, broadcast events
Realtime engine	Clients	WebSocket (Reverb)	Broadcast events
Backend	ML service	HTTP REST / JSON	Text to classify, recommendations
Sharing & Export	Social APIs	HTTPS	Post content to publish
Desktop sync agent	Sync API	HTTPS	Change sets and queued actions

Table 6. 4 Table for Internal interfaces

6.6 Hardware and software interfaces

Hardware interfaces. The system has no specialized hardware interfaces. Clients run on ordinary personal computers, laptops or phones (web) and on desktop or laptop machines (Java client). Servers run on commodity or cloud hardware and the file store uses ordinary disk storage.

Software interfaces. The backend interfaces with the PHP runtime and Laravel framework, database drivers for MySQL or PostgreSQL, a Redis client library and an SMTP library. The desktop client interfaces with the Java runtime and an SQLite driver. The ML service interfaces

with the Python runtime, FastAPI and scikit-learn. Social-media integration uses each platform's HTTPS API with an authorized application credential.

6.7 Communication protocols

- HTTPS with REST and JSON for ordinary client–server requests and for the ML service.
- Secure WebSocket (WSS) for real-time events.
- The sync API over HTTPS for offline change exchange.
- SMTP for outbound e-mail.

7. Security Design

7.1 Authentication

The system uses Laravel Authentication to verify the identity of users before granting access to protected resources. Each user is required to provide a valid email address and password during login. Passwords are never stored in plain text instead, they are securely hashed using the crypt hashing algorithm before being stored in the database.

Authentication mechanisms implemented in the system include secure user registration and login, password hashing using crypt, session management for authenticated users, password reset functionality through email verification and automatic session termination after logout.

These mechanisms ensure that only legitimate users can access the platform.

7.2 Authorization and Access Control

The Smart Discussion Forum implements Role-Based Access Control (RBAC) to restrict access to system resources based on user roles. Three primary roles are supported that is administrator, lecturer and student.

Each role is granted specific permissions according to its responsibilities. For example, administrators can manage users and system settings, lecturers can create quizzes and evaluate participation, while students can participate in discussions and complete quizzes.

Authorization checks are performed before executing sensitive operations to prevent unauthorized access.

7.3 Secure Communication

All communication between clients and the server is secured using HTTPS. HTTPS encrypts data transmitted across the network and protects users from eavesdropping, man-in-the-middle attacks and data tampering.

7.4 Data Protection

The system protects sensitive information through multiple security measures.

These include:

- Password hashing before storage.
- Input validation and sanitization.
- Secure file upload validation.
- Restricted access to sensitive database records.

8. User Interface Design

User Interface (UI) Design is the process of designing the visual and interactive elements of a software application, website, or system. It focuses on how users interact with the system and aims to create an interface that is attractive, easy to use, efficient, and accessible

The following are the web interface design of Student Discussion Forum system

8.1 Login interface

welcome to
smart discussion forum
signin to continue to you account

Email address:
stevenoumo5@gmail.com

Password: [Forgot password?](#)

☒ Remember

☒ I agree to the [terms and conditions](#) and [privacy policy](#)
You must agree to our terms and conditions to access the system

Sign in

Don't have an account? [sign up](#)

figure 8. 1: Diagram of the login screen

Figure 8.1 shows the login dashboard. When the user enters the Email and password, he/she must agree to the terms and condition to before clicking the sign in button. On clicking the sign in button, the Email and Password are checked in the database to confirm if they are correct. If they are wrong the user is prompted to either sign up or forgot password options in order to capture user credential.

8.2 Student Dashboard

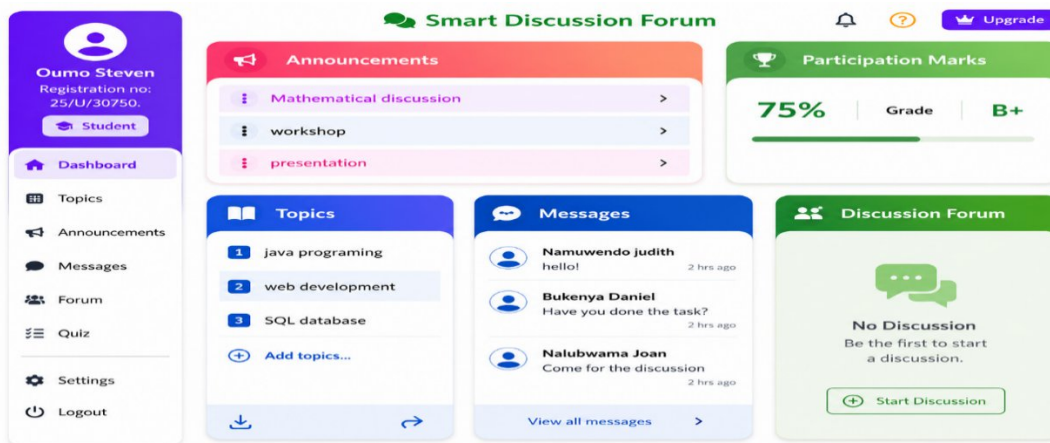


figure 8. 2: Diagram of the student dashboard screen

Figure 7.2 shows the student dashboard screen. When the user logs in as a student, the system directs him/her to the Student Dashboard, which displays the student's profile details alongside the current Participation Marks and grade computed from their forum activity. The dashboard presents Announcements posted by the lecturer in the order they were published. The student can navigate using the side menu to access Topics, where he/she can view existing discussion topics or add a new topic by clicking "Add topics." The Messages panel displays direct messages received from other group members, showing the sender's name, message preview and time sent. The Forum panel shows ongoing discussions the student is part of, while the Quiz menu allows the student to access scheduled quizzes published by the lecturer

8.3 Lecturer Dashboard

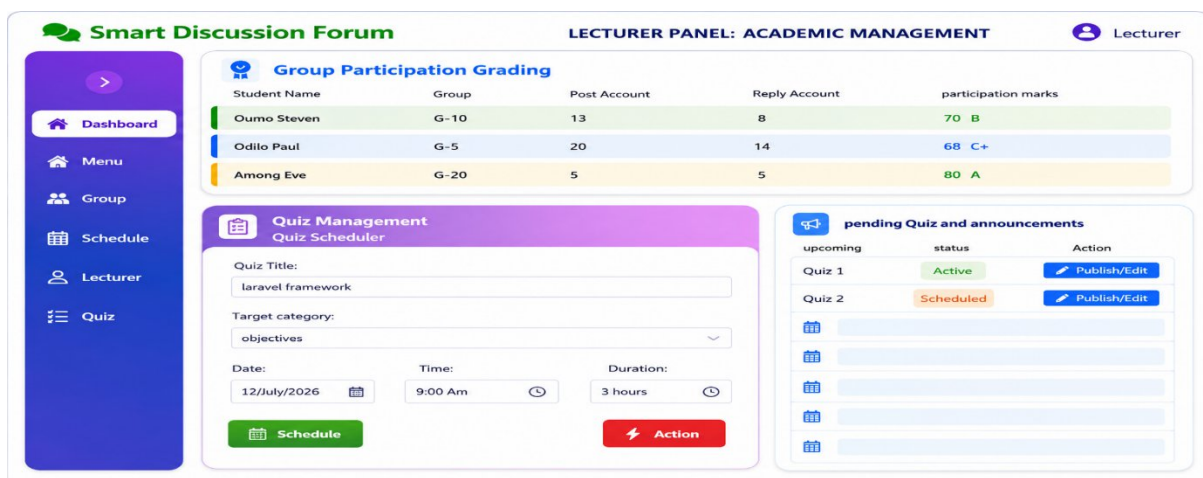


figure 8. 3: Diagram of the lecture dashboard screen

Figure 8.3 shows the lecturer dashboard screen. When the user logs in as a Lecturer, the system directs him/her to the Lecturer Panel, where two main functions are available: Group Participation Grading and Quiz Management. In the Group Participation Grading section, the system automatically retrieves each student's post account and reply account from the discussion forum database and computes a participation mark and corresponding grade based on the student's activity within their assigned group.

In the Quiz Management section, the lecturer can schedule a new quiz by clicking "Schedule" to save it or "Action" to cancel/modify it. Once scheduled, the quiz appears in the "Pending Quiz and Announcements" table with its current and the lecturer can choose to Publish or Edit it before it becomes visible to students. This panel therefore allows the lecturer to monitor student engagement and manage academic assessments from a single interface

8.4 Admin Dashboard

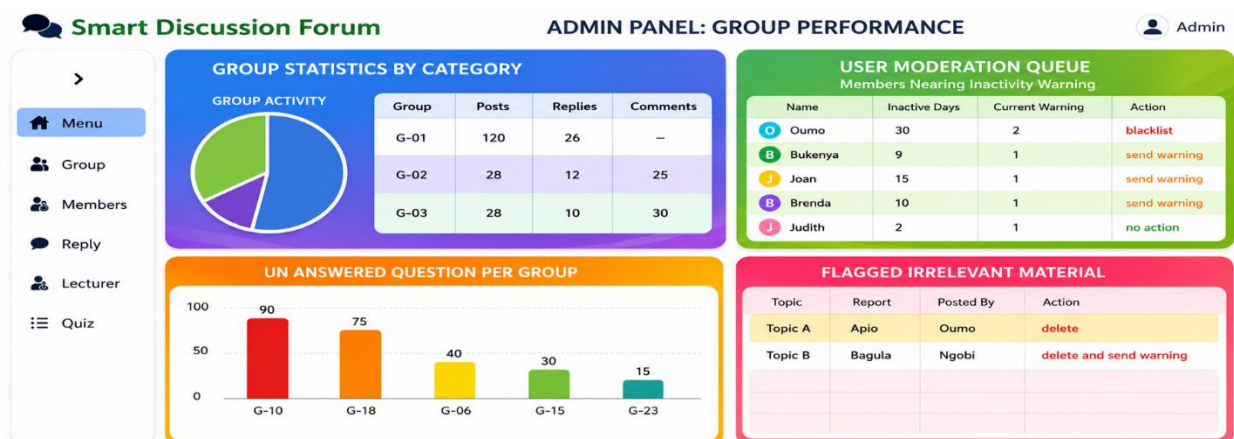


figure 8. 4: Diagram of the Admin dashboard screen

Figure 8.4 shows the Administrator dashboard screen. When the user logs in as administrator, he can view the group statistics, user moderation queue, unanswered questions per group and flagged irrelevant material.

9. Deployment architecture

9.1 Overview

The deployment architecture describes how the Smart Discussion Forum system components are physically deployed and interact within the production environment. The system follows a client-server deployment model where multiple client applications communicate with a centralized backend server through secure internet connections. The deployment architecture supports both web-based and desktop-based access while ensuring scalability, maintainability and efficient resource utilization.

9.2 Deployment Environment

The major deployment nodes include:

- Web Clients (Browsers)
- Java Desktop Clients
- Laravel Application Server
- MySQL Database Server
- Redis Cache Server
- Machine Learning Service Server
- File Storage Service
- Email Service Provider
- Social Media Service APIs

9.3 Risk register

Risk	impact	Mitigation
ML model inaccuracy	Medium	Retrain model regularly
Database failure	High	Daily backups
Network outage	Medium	Offline synchronization
Security attacks	High	Authentication and Role based access control

Table 9. 2: Risk register

9.4 Deployment diagram

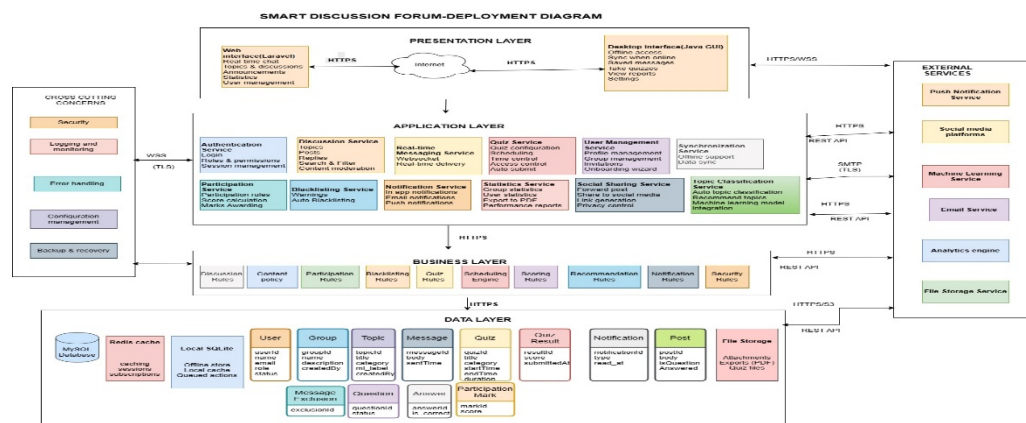


figure 9. 1: Deployment diagram of the discussion forum

9.5 Deployment Components

Client Layer

The client layer consists of users accessing the system through either a web browser or a Java desktop application.

Web Client

The web client is implemented using Laravel Blade and Livewire interfaces. Users access the application through modern web browsers such as Google Chrome, Mozilla Firefox and Microsoft Edge.

Desktop Client

The Java desktop application provides offline functionality by storing selected data in a local SQLite database. When connectivity is restored, synchronization services exchange updates with the central server.

Application Server

The application server hosts the Laravel backend and contains the core business logic of the system. It manages user authentication, discussions, quizzes, notifications, participation tracking, synchronization services and communication with external services.

Database Server

The MySQL database serves as the primary persistent storage for the system. It stores information related to users, groups, topics, posts, messages, quizzes, participation marks and notifications. The database acts as the authoritative source of system data.

Redis Server

Redis provides high-speed in-memory storage for application caching, session management, subscribe messaging and real-time event broadcasting. Redis improves system performance and reduces database workload.

Machine Learning Server

The Machine Learning Service is deployed separately using Fast API and Python. It performs topic classification, content moderation and topic recommendation. Separating the ML service improves maintainability and allows independent scaling.

File Storage Service

The file storage service manages uploaded attachments, exported PDF documents, backup files and shared resources. The database stores file references while the actual files remain in dedicated storage.

These protocols ensure secure, reliable, and efficient communication across all system components.

10. Glossary of Terms

Term	Definition
API	Application Programming Interface; a defined set of operations the backend exposes to clients.
FastAPI	A Python web framework used to build the machine-learning service.
Laravel	A PHP web framework used for the backend and the web client.
Laravel Reverb	Laravel's WebSocket server, used for real-time broadcasting.
Lockdown mode	A restricted quiz interface that disables other actions until the attempt is submitted.
Machine learning (ML)	Techniques that learn patterns from data, used here for topic classification and recommendation.
Moderation	Automatic filtering of irrelevant or flooded posts to keep discussions focused.
MVC	Model-View-Controller; the pattern organising request handling in the backend.
Offline-first	A design in which the desktop client works without connectivity and synchronises later.
Onboarding	The process of joining a group and agreeing to its rules before participating.

Table 10. 1: Table of Glossary of Terms